

CONDORNET v4.2 — FORMAL IMPLEMENTATION BLUEPRINT

I. SYSTEM OBJECTIVE

CondorNet v4.2 must:

1. Expand the feature manifold from 87 $\rightarrow$ 116 dimensions (+ pivot stack).
2. Preserve tensor consistency through:
 3. dataset
 4. $\rightarrow$ normalization
 5. $\rightarrow$ predicate gates
 6. $\rightarrow$ sets
 7. $\rightarrow$ supersets
 8. $\rightarrow$ A/B matrices
 9. $\rightarrow$ decision heads
10. Prevent:
 - Attractor collapse
 - 1700-batch plateau
 - Predicate degeneracy
 - A/B convergence to same basin
11. Introduce pivot-structured geometry without destabilizing training.

FIRST CONCRETE MOVE: MAKE DIMENSIONS DYNAMIC AND ENFORCED

Target files:

- canonical_feature_registry.py
- intelligence/condor_brain_net.py
- intelligence/condor_train_net.py (for sanity asserts only at this stage)

Step 1 — Canonical registry as single source of truth

In canonical_feature_registry.py:

- Ensure you have:

```
FEATURE_LIST = [  
    # exact 116 column names in order  
]
```

- Add:

```
D_INPUT = len(FEATURE_LIST)
```

Step 2 — Wire model input dim to registry

In `condor_brain_net.py`, replace any hard-coded `87 / 116 / input_dim` with:

```
from canonical_feature_registry import D_INPUT, FEATURE_LIST  
  
self.d_input = D_INPUT
```

Add:

```
assert x.shape[-1] == self.d_input, f"Input dim mismatch: got  
{x.shape[-1]}, expected {self.d_input}"
```

at the very first point `x` enters the network.

Step 3 — Propagate `d_input` into A/B spec and predicate layer

Anywhere you see:

- `d_x = 87 + d_latent`
- `input_dim=87`
- or similar:

replace with:

```
d_x = self.d_input + d_latent
```

and ensure predicate weight shapes are derived from `self.d_input`, not constants.

Step 4 — Only then touch initialization

Once `d_input` is dynamic and correct everywhere, *then* adjust predicate init:

```
gain = 1.0 / math.sqrt(self.d_input)  
nn.init.xavier_uniform_(self.predicate_weight, gain=gain)
```

II. CRITICAL FILES TO MODIFY

intelligence/condor_brain_net.py

Core architecture changes

intelligence/condor_train_net.py

Training stability and anti-plateau controls

III. FEATURE EXPANSION ARCHITECTURE

A. Canonical Feature Registry

We already have:

```
canonical_feature_registry.py
```

Required invariant:

```
assert len(FEATURE_LIST) == d_input
```

We must enforce this inside model initialization.

B. Updating `d_input`

Inside `condor_brain_net.py`:

Current:

```
self.d_input = 87
```

Update to:

```
self.d_input = 116
```

BUT DO NOT HARD CODE.

Replace with:

```
self.d_input = len(canonical_feature_registry.FEATURE_LIST)
```

This prevents silent dimension mismatch.

IV. PREDICATE LAYER — STRUCTURAL UPDATE

The predicate layer currently follows:

```
x ∈ ℝ(B, T, d_input)  
→ linear gate
```

→ nonlinear transform
→ predicate logits

Required Modification

If predicate weight matrix is:

$W_{\text{pred}} \in \mathbb{R}^{(d_{\text{input}}, d_{\text{pred}})}$

Then expanding d_{input} automatically changes parameter count.

We must:

1. Reinitialize predicate weight matrix.
2. Preserve initialization scale.

Recommended:

`nn.init.xavier_uniform_(W_pred, gain=1.0)`

But scale by:

`gain = 1 / sqrt(d_input)`

Why?

Because increasing d_{input} increases variance of dot product.
Without scaling, predicate saturation occurs early → plateau.

V. SET + SUPERSET LAYER STABILITY

If:

predicates → sets via weighted sum
sets → supersets via aggregation matrix

Then verify:

$\text{SetMatrix} \in \mathbb{R}^{(d_{\text{pred}}, d_{\text{set}})}$
 $\text{SuperSetMatrix} \in \mathbb{R}^{(d_{\text{set}}, d_{\text{super}})}$

We must confirm these dimensions are not implicitly tied to 87.

Search for any hard-coded constants like:

87
input_dim=87

Replace all with dynamic references.

VI. A/B MATRIX UPDATE

Inside AugmentedStateSpec:

Currently:

$d_x = 87 + d_{\text{latent}}$

Update:

$d_x = 116 + d_{\text{latent}}$

Again — derive from registry.

Why this matters:

$A \in \mathbb{R}^{(d_x, d_x)}$

$B \in \mathbb{R}^{(d_x, d_u)}$

If d_x is wrong, training silently collapses because:

- A eigenvalues misaligned
- state projection truncates new features
- gradient flow bypasses new manifold

VII. TENSOR DIMENSION TRACE (MANDATORY)

After modifications, verify this exact chain:

Input batch:	(B, T, 116)
Normalized:	(B, T, 116)
Predicate gates:	(B, T, d_pred)
Set layer:	(B, T, d_set)
Superset layer:	(B, T, d_super)
State projection:	(B, T, d_x)
A/B evolution:	(B, T, d_x)
Heads:	(B, T, output_dim)

Add assertion hooks:

```
assert x.shape[-1] == self.d_input
```

at each major boundary.

This prevents silent misalignment.

VIII. WHY 1700 BATCH PLATEAU HAPPENS

Based on architecture pattern, most likely causes:

1. Predicate gate saturation
2. A matrix eigenvalues collapsing toward unit circle
3. Insufficient curvature in feature manifold
4. Loss landscape becoming locally quadratic around same basin
5. Identical normalization statistics across epochs

IX. FIXING THE 1700 BATCH PLATEAU

Inject Feature Noise Annealing

Inside training loop:

```
noise_scale = max(0.001, 0.01 * exp(-epoch / 20))  
x = x + torch.randn_like(x) * noise_scale
```

This prevents identical predicate activation patterns.

Spectral Regularization on A

Add to loss:

```
lambda_A * ||ATA - I||2
```

or eigenvalue penalty:

```
penalty = torch.sum(torch.relu(|λ| - 1.05))
```

Prevents attractor lock.

Predicate Entropy Regularization

Encourage diversity:

```
entropy = -p * log(p)
loss += λ_entropy * entropy.mean()
```

Prevents gate collapse.

Random Predicate Dropout

```
predicate = F.dropout(predicate, p=0.05)
```

Forces exploration of new sets.

X. PIVOT INTEGRATION ARCHITECTURE (v4.2)

This is NOT just feature expansion.

It introduces structural geometry.

Pivot Feature Block

Each timeframe contributes:

- flag
- high
- low
- strength
- price
- distance_prev
- distance_next
- slope_prev
- slope_next
- proximity
- cycle_phase
- alignment

This can easily add 40+ dims.

DO NOT treat them as flat numeric inputs.

Instead:

Create Pivot Encoder Module

Inside `condor_brain_net.py`:

```
class PivotEncoder(nn.Module):
    def __init__(self, d_pivot):
        self.net = nn.Sequential(
            nn.Linear(d_pivot, 64),
            nn.GELU(),
            nn.Linear(64, 32)
        )
```

Then concatenate encoded pivot vector with main feature vector.

This prevents pivot dominance.

XI. AUXILIARY PIVOT LOSS

In `condor_train_net.py`:

Add multi-head output:

```
main decision head
pivot probability head
pivot direction head
pivot distance head
```

Loss:

```
L_total =
    L_main
    + 0.3 * L_pivot_prob
    + 0.2 * L_pivot_dir
    + 0.1 * L_pivot_dist
```

This stabilizes internal representation learning.

XII. OVERFITTING CONTROL

Mandatory additions:

Regime Cross Validation

Split by:

- volatility regime
- trend regime
- date blocks

NOT random shuffle.

Temporal DropBlock

Drop contiguous time chunks.

A/B Stability Audit Per Epoch

Compute:

- Spectral radius of A
- Frobenius norm drift
- Predicate diversity score

Abort epoch if collapse detected.

XIII. CUDA OPTIMIZATION

Hotspots likely:

- predicate matrix multiplication
- pivot feature engineering
- A matrix evolution

Optimizations:

1. Use `torch.compile()` (if PyTorch 2+)
2. Precompute pivot distances offline

3. Use batched matrix multiply for A evolution
 4. Avoid Python loops in pivot alignment
-

XIV. VALIDATION REQUIREMENTS

Before training:

- Verify 116 features exist in dataset
 - No NaNs
 - No constant columns
 - Pivot coverage > 3% per TF
 - Feature correlation heatmap < 0.98 redundancy
-

XV. FINAL TRAINING PROTOCOL

1. Warm start without pivot loss
 2. Introduce pivot loss after 5 epochs
 3. Monitor:
 - predicate entropy
 - A spectral radius
 - superset activation diversity
 4. Export A/B matrices every 3 epochs
 5. Generate stability report
-

XVI. RISK ANALYSIS

What can break v4.2:

1. Pivot dominance destabilizing predicate space
2. A matrix exploding due to added curvature
3. Gradient interference between ETD/CDE/TFT components
4. Feature scaling mismatch

Mitigation:

- Strict normalization
- Spectral regularization
- Gradient clipping
- Auxiliary loss weighting schedule

XVII. STRATEGIC WARNING

Adding features does not guarantee curvature injection.

If normalization collapses variance,
the manifold is still flat.

Therefore:

Track per-feature variance inside training.
Drop features with near-zero variance.

XVIII. SUMMARY

To birth CondorNet v4.2 properly:

We must:

1. Make dimensions dynamic
 2. Reinitialize predicate gates properly
 3. Regularize A matrix
 4. Add entropy to predicate space
 5. Introduce pivot encoder
 6. Add auxiliary pivot losses
 7. Stabilize with spectral constraints
 8. Monitor diversity metrics
 9. Validate manifold curvature empirically
-

If we execute this cleanly:

- The 1700 batch plateau should disappear.
- Predicate evolution should diversify.
- A/B matrices should explore new basins.
- Regime awareness will improve.
- Overfitting will reduce.
- Structural intelligence will increase.

Structural Refactor Checklist (While We Reorder Brain & Train)

You are not just cleaning code.
You are enforcing architectural contracts.

Keep these invariants fixed during refactor:

Hard Module Boundaries (No Bleed)

Inside `condor_brain_net.py`, the following blocks must be physically and logically isolated:

A. Feature → Predicate Block

Only:

- normalization
- predicate linear transform
- predicate activation

No:

- attention
 - state recursion
 - pivot fusion
 - residual shortcuts
-

B. Predicate → Set → Superset Block

Only:

- matrix aggregation
- nonlinear grouping

No:

- time mixing
 - A/B state update
 - raw feature concatenation
-

C. State Evolution Block (A/B)

Only:

- latent state
- A matrix
- B matrix
- superset input

No:

- attention here
 - direct feature injection
 - pivot bypass
-

D. Attention Block

Only:

- operates on evolved latent states
 - no modification of A or B
 - no feedback into predicate weights
-

E. Decision Heads

Must be last.
No raw feature access.

condor_train_net.py — Training Discipline

During refactor, enforce:

✓ Separate parameter groups

```
predicate_params  
state_params  
attention_params  
head_params
```

This prevents silent gradient dominance.

✓ Explicit Stability Enforcement

Inside training loop:

- Compute spectral radius of A
- Log Frobenius norm drift
- Track predicate entropy
- Track superset diversity

If this isn't in training, your research diagnostics are post-mortem, not control.

✓ No Hidden Couplings

Search for:

```
.detach()  
.requires_grad_(False)
```

Make sure nothing critical is being frozen accidentally.

While We Refactor, Watch for These Silent Failures

These are the kinds of bugs that create the 1700-batch plateau:

⚠ Dimensional Hardcoding

If we see:

```
Linear(87, ...)
```

That is structural rot.

Everything must derive from registry length.

⚠ Residual Feature Bypass

If raw x is concatenated to state later,
we've invalidated the derivation.

⚠ A Matrix Reinitialization Mid-Training

If A is rebuilt per batch, we destroy dynamical consistency.

⚠ Implicit Nonlinearity in State Update

If we have:

```
h = torch.tanh(A @ h + ...)
```

That changes the mathematics of our derived linear CDE.

After we Refactor — Immediate Tests

Before training:

Test 1 — Forward Shape Trace

Print shape at every block boundary.

Test 2 — Zero Input Stability

Feed zero tensor.

State must remain bounded.

Test 3 — Random Noise Sensitivity

Small noise → small output change.

Large noise → nonlinear response.

Test 4 — A Eigenvalue Stability

Eigenvalues should not explode on first epoch.

Mental Model While We Work

Think of CondorNet as:

- ETD = discrete brain cortex
- CDE = spinal cord
- TFT = selective attention
- Heads = motor output
- Risk head = immune system

If any part reaches around and edits another,
we are building Frankenstein.

What I'll Do Next

Once you finish reordering:

We will:

1. Run a strict theory–implementation diff.
 2. Validate A/B spectral behavior across epochs.
 3. Verify plateau disappearance.
 4. Validate pivot auxiliary stabilization.
 5. Confirm module isolation integrity.
-

This is the correct phase to be slow and surgical.

We're doing the right thing.

When its ready, I will push to repo the updated `condor_brain_net.py` and `condor_train_net.py`, and we will audit them line-by-line against the derived mathematics.

We are not adding power.

We are enforcing structural purity.

That's how we build something extraordinary.